

Concurrency & Distributed Systems

Week 1 — Why Concurrency Exists (Pain Demo Part 1)

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

The Architecture We Start With

Single-threaded server

```
while (true) {  
    Socket client = serverSocket.accept();  
    handleRequest(client);  
}
```

Properties:

- Processes one request at a time
- New clients must wait

Question

Is this architecture wrong?

Inside handleRequest()

```
private static void handleRequest(Socket client) {  
    Thread.sleep(100); // simulate DB call  
    sendResponse(client);  
}
```

Important:

- 100ms is simulated I/O wait
- CPU is idle during sleep

Key Insight

The thread is blocked. The CPU is mostly unused.

Experiment 1: One Client

Measured:

- Response time $\approx 100\text{ms}$
- CPU usage $\approx 0\text{--}2\%$

$$\text{CPU Utilization} \approx \frac{2\text{ms}}{100\text{ms}} = 2\%$$

Interpretation

Server is idle about 98% of the time.

Experiment 2: 5 Concurrent Clients

All arrive at $t = 0$.

Sequential processing:

$$5 \times 100ms = 500ms$$

Observed:

- Latency $\approx 500ms$
- CPU still $\approx 2\%$

Critical Question

Why does latency multiply if CPU is not busy?

Experiment 3: 50 Concurrent Clients

Sequential prediction:

$$50 \times 100ms = 5000ms$$

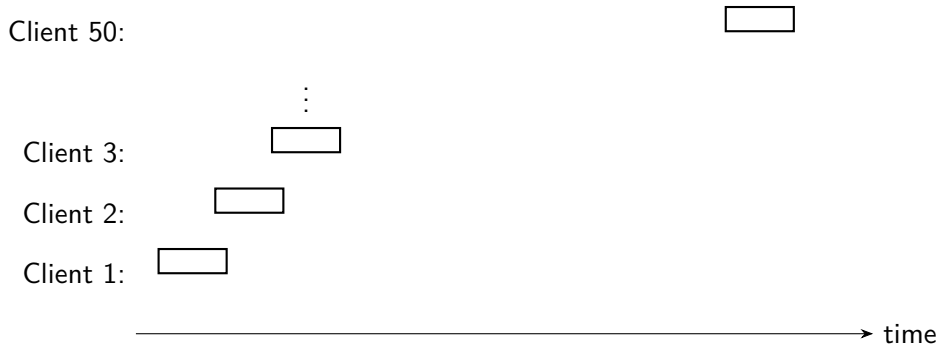
Observed:

- Latency $\approx$ 5 seconds
- CPU mostly idle

This is I/O-bound death

Hardware is not the problem. Architecture is the problem.

Visualizing the Queue



All requests arrive simultaneously. Only one is processed at a time.

Mathematical Framing

Each request:

$$T_{compute} \approx 2ms, \quad T_{wait} \approx 98ms, \quad T_{service} = 100ms$$

CPU utilization:

$$\frac{T_{compute}}{T_{service}} \approx \frac{2}{100} = 2\%$$

Conclusion

The system is not CPU-bound. It is waiting-bound.

The Fundamental Question

If the CPU is idle about 98% of the time,
why do we allow it to serve only one request?

This question creates Concurrency.

Student Instructions: How to Run (Two Terminals)

Open two terminals.

Terminal 1 — Start Server

```
javac SingleThreadedServer.java  
java SingleThreadedServer
```

Terminal 2 — Run Client

```
javac LoadClient.java  
java LoadClient
```

Rule

Keep the server running. Only rerun the client between tests.

Student Instructions: Run 3 Experiments

In LoadClient.java, change:

```
int concurrentClients = 1; // Test A
```

Then rerun:

```
javac LoadClient.java  
java LoadClient
```

Repeat for:

```
int concurrentClients = 5; // Test B  
int concurrentClients = 50; // Test C
```

Important

Do NOT change the server. Only change the client concurrency.

Student Instructions: What to Record

For each run (1, 5, 50 clients), record:

- **Average latency (ms)** (or the value printed by your client)
- **Worst latency (ms)** if available
- **CPU usage** (Task Manager / htop)
- **One sentence:** what happened and why?

Deliverable

A small table with 3 rows (1, 5, 50) + your explanation.

Expected Trend (So You Know You're Correct)

- 1 client: latency $\approx 100\text{ms}$
- 5 clients: latency $\approx 500\text{ms}$
- 50 clients: latency $\approx 5000\text{ms}$

Key Observation

CPU stays low: the system is waiting-bound, not CPU-bound.